



PRACTICE 2: IMPLEMENTATION OF A 3-TIER APPLICATION WITH MAVEN AND GIT

1. Objectives

- Become familiar with the use of Maven and Git in the process of building an application.
- Practice implementing applications with a three-tier architecture.

2. Activities

IMPORTANT: The use of Git and Maven during the practical exercise must be carried out in accordance with the instructions in the document "Instructions for the use of Git and Maven in practical exercises", available in Moodle, in addition to the instructions given in this same statement.

The activities to be carried out in practice are the following:

- Analyze the application to be developed, for which the following should be consulted:
 - The application's UML model, available on Moodle as a StarUML file, includes: use case and domain class diagrams, architecture diagram, detailed design (class) diagram, and deployment diagram. Screenshots of the diagrams are also included in an appendix to the assignment.
 - The application description is available in the annexes to the statement.
- Based on the provided deployment model, create the project structure Maven corresponding (for now only the structure, without code).
- Download and extract the CodigoPractica2.zip file available on Moodle. This file contains the code (in whole or in part) for the application's classes and interfaces, with the exception of the GestionTiendas and GestionEmpleados classes. The student must then:
 - Add each class to the corresponding Maven project (again, according to the (deployment diagram provided).
 - Create the skeleton of the GestionTiendas and GestionEmpleados classes (header, attributes and unimplemented methods) and add it to the corresponding project.
 - Finally, compile all projects.

NOTE: Apply Maven's standard criteria for naming and organizing code in Java packages.

IMPORTANT. After completing this step, commit the changes to your local Git repository with the "Initial version" message and upload the changes to the remote repository.

- Complete the code of the Employee class and the StoreManagement classes according to their documentation (see analysis of the rest of the interfaces), the description of the EmployeeManagement application provided and the provided code.



NOTE: Date attributes have been defined as type `LocalDate` because this greatly simplifies development compared to using the traditional Java Date class. A tutorial on the basics of using this class is available at the following link:

<http://tutorials.jenkov.com/java-date-time/localdate.html>

- e) Create the self-contained `.jar` file corresponding to the complete application and run it from the command line. It is sufficient to verify that the application launches correctly, i.e., that its interface appears, as its validation will be carried out in the next exercise.

3. Evaluation Criteria and Clarifications

A compressed file containing the following elements must be submitted:

- A `.zip` file containing all the developed Maven projects, properly exported.

Export projects from Eclipse:

File => Export => Archive File => Choose the projects to export (removing the target folder and Eclipse's own files) => Finish

- The `.jar` file corresponding to the complete application.

Patricia López Martínez

Juan Rivas Concepción

Adolfo Garandal



APPENDIX. DESCRIPTION OF THE CASE STUDY

It is an application that manages the employees of a store franchise.

Detailed specification of use cases

Use case: New store setup

Actor: Manager

Sequence of steps:

1. The manager selects the "New Store" option.
2. The system asks for the store's details: name and address.
3. The manager enters the data and selects "Confirm".
4. The system adds the new store.
 - a. If a store with the same name already exists, the error is reported.

Use case: Low store

Actor: Manager

Sequence of steps:

1. The manager selects the "Close Store" option.
2. The system asks for the store name.
3. The manager enters the data and selects "Confirm".
4. The system removes the store.
 - a. If the store does not exist, the error is reported.
 - b. If the store has employees, it is communicated and not removed.

Use case: Store inquiry

Actor: Manager

Sequence of steps:

1. The manager selects "Check Stores".
2. The system displays the data of the franchise stores (name, address and number of employees)

Use case: Store inquiry

Actor: Manager

Sequence of steps:

3. The manager enters the name of a store in the "Store Name" field.
4. The system displays the store's data (name, address, employee list, and the total monthly expenditure on employee salaries) a. If the store does not exist, the error is reported.

Use case: New employee

Actor: Manager and HR Manager

Sequence of steps:

1. The user selects the "New Employee" option.
2. The system requests the employee's details (name, ID number, and category) and the name of the store.
3. The user enters the data and selects "Confirm".
4. The system adds the new employee to the indicated store.
 - a. If the store does not exist, the error is reported.
 - b. If the employee already exists, the error is reported.



Use case: Terminate employee Actor:

Manager and HR Manager Sequence of

steps: 1. The user selects

the "Terminate Employee" option.

2. The system asks for the employee's ID and the name of the store where they work.

3. The manager enters the data and selects "Confirm".

4. The system deletes the employee. a. If

the employee does not exist, an error is reported. b. If a store

with the specified name does not exist, an error is reported.

c. If the employee does not belong to the indicated store, the error is reported.

Use case: Move employee

Actor: Manager

Sequence of steps: 1.

The manager selects the "Move Employee" option.

2. The system asks for the employee's ID number, the name of the current store, and the name of the destination store.

3. The manager enters the data and selects "Confirm".

4. The system adds the employee to the destination store, removing them from the current one.

a. If the employee does not exist, the current store or the destination store is communicated mistake.

b. If the employee is not listed in the current store, the error is reported.

Use case: Register medical discharge

Actor: HR Manager Sequence

of steps: 1. The HR

manager selects "Manage medical leaves"

2. The HR manager enters the employee's ID number.

3. The system displays the employee's information and an option to register them. a. If the employee does not exist, an error message is displayed. b. If the employee is not currently on leave, the registration option does not appear.

4. The HR manager selects "Report new hire".

5. The system updates the employee's status

Use case: Registering sick leave Actor: HR

Manager Sequence of steps: 1.

The HR manager selects

"Manage sick leave"

2. The HR manager enters the employee's ID number.

3. The system displays the employee's information and an option to terminate their employment.

a. If the employee does not exist, an error message is

displayed. b. If the employee is already terminated, the termination option does not appear.

4. The HR manager selects "Report termination".

5. The system updates the employee's status.



Rules for calculating an employee's salary

An employee's salary is calculated according to the following rules:

- Salary is the sum of the base salary plus the seniority bonus. • The base salary depends on the employee's category, according to the values of the Table 1.
- The seniority supplement is calculated according to the data in Table 2.
- If the worker is on sick leave, the salary is reduced by 25%.

Table 1. Base salary according to category

Post	Base salary
IN CHARGE	2000
SELLER	1500
ASSISTANT	1000

Table 2. Seniority Supplement

Antiquity	Complement
> 5 years	50
> 10 years	100
> 20 years	200

Graphic interface

Currently, only the functionality for the Store Inquiry use case is partially implemented. Through an interface like the one shown in Figure 1, the manager enters the name of the store to be inquired about:

- If the store exists, its address is shown, a list with the name and surname of each employee, and the total monthly cost of salaries for the store.
- If the store does not exist, "Store Does Not Exist" is displayed in the Address field and the rest is Leave empty.

Figure 1. Graphical interface of the Store Inquiry use case

APPENDIX 2. IMPLEMENTATION OF THE PERSISTENCE LAYER

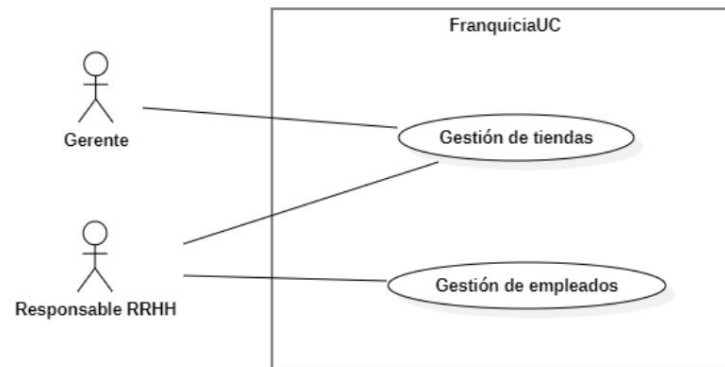
The persistence layer, or DAO layer, is implemented using H2, a Java-based database that can be configured to be used as an in-memory database, which is how it has been used in this application. This means that the data only exists for the lifetime of the application.

The initial data loading into the database is done from within the code itself.

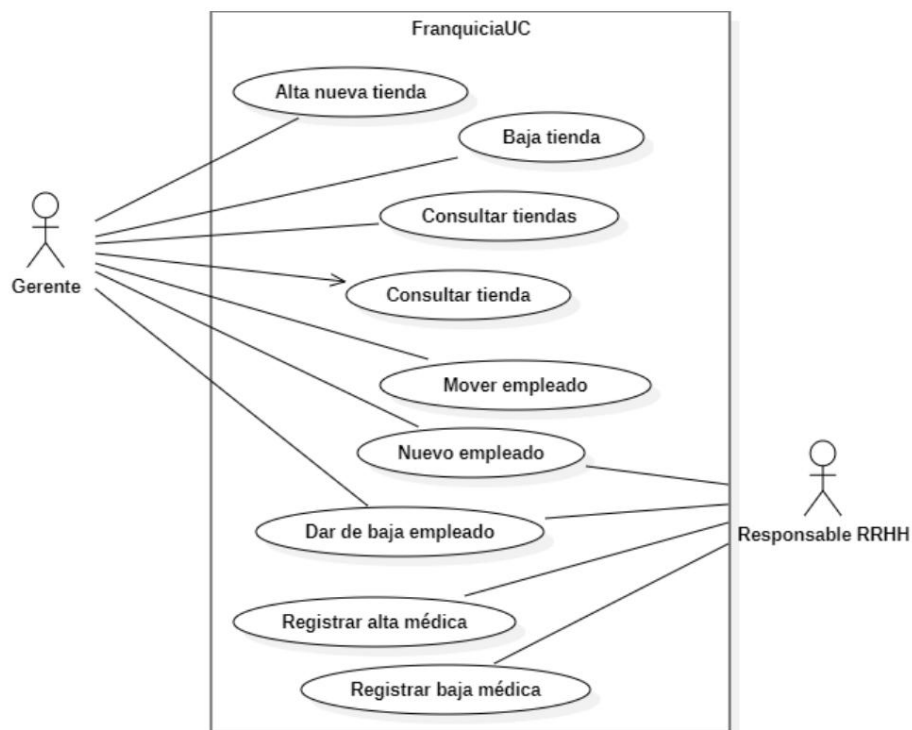


APPENDIX 3. UML DIAGRAMS OF THE CASE STUDY

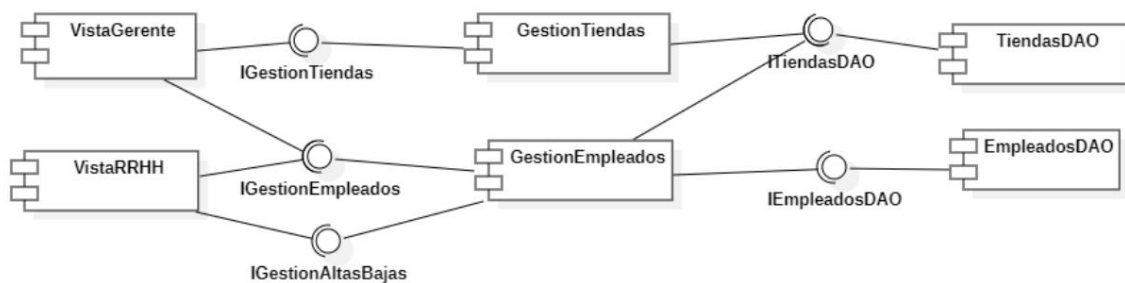
Use cases - General



Use cases – Detailed

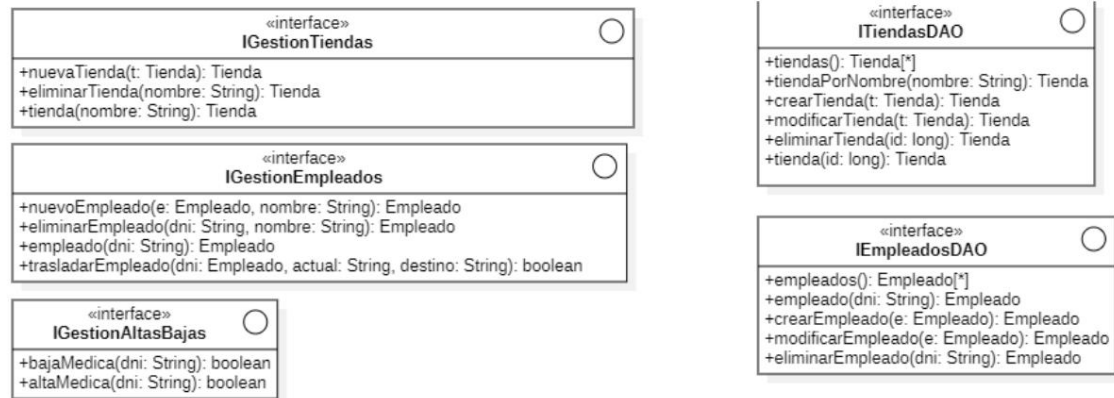


Architecture - Components

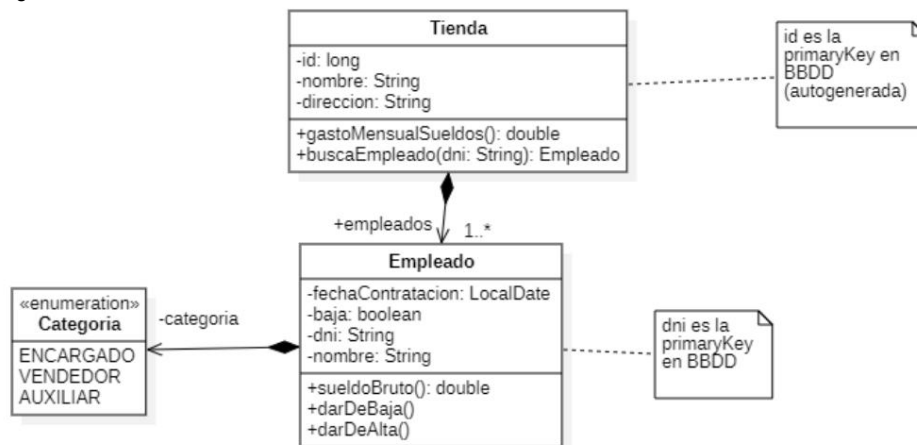




Architecture – Interfaces



Detailed design



Deployment

